Mise en place de la base de données

1. Présentation

Nous allons utiliser le projet 'database' à ouvrir sous PhpStorm. Ce projet comportant les répertoires suivants :

Sql	Script SQL permettant la création de la base de données, des tables, des vues, des procédures et des tests
Importation	Script Php permettant d'importer les données dans les tables à partir d'API ou de fichiers JSON placés dans le sous répertoire data
Include	Contient le script fonction.php inclus dans les scripts PHP du répertoire Importation. Il contient la fonction permettant la connexion à la base de données GSB et des fonctions génériques permettant le traitement des fichiers json et la mise en place d'une transaction.
Sauvegarde	Script sauvegarder.php réalisant une sauvegarde horodatée de la base de données stockée dans le sous répertoire backup
Utilisateur	Script create.php permettant de créer un utilisateur MySQL pour chaque visiteur avec ses droits. L'id du visiteur correspond au nom de l'utilisateur, le mot de passe de l'utilisateur correspond à la date d'embauche du salarié au format aaaaammjj

2. Création des tables

Étude du script Sql/create.sql

Création de la base de données

Création des tables departement, ville, famille, medicament, specialite, typepraticien, praticien, visiteur, motif, visite, echantillon

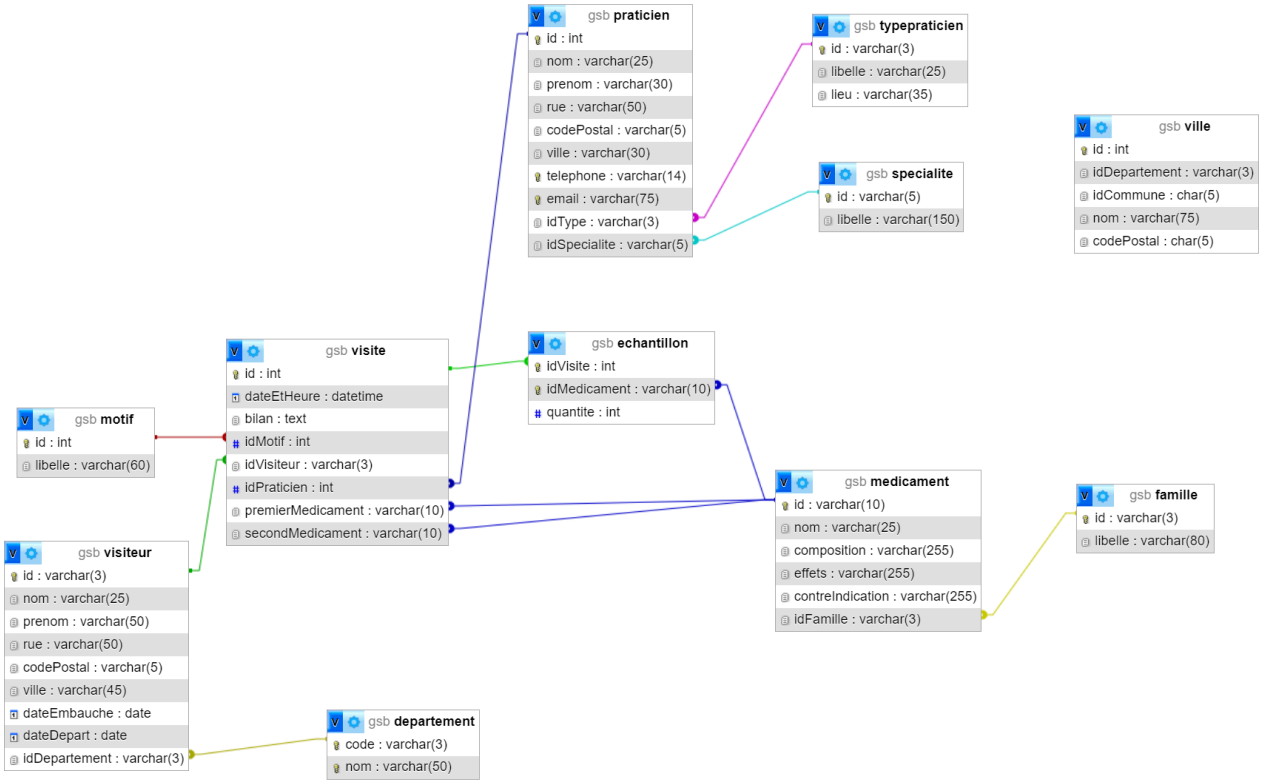
Il comporte aussi le trigger avantAjoutVisiteur qui vérifie l'unicité du visiteur au niveau d'un département

1. Lancer le script create.sql et vérifier le résultat attendu présenté ci-dessous

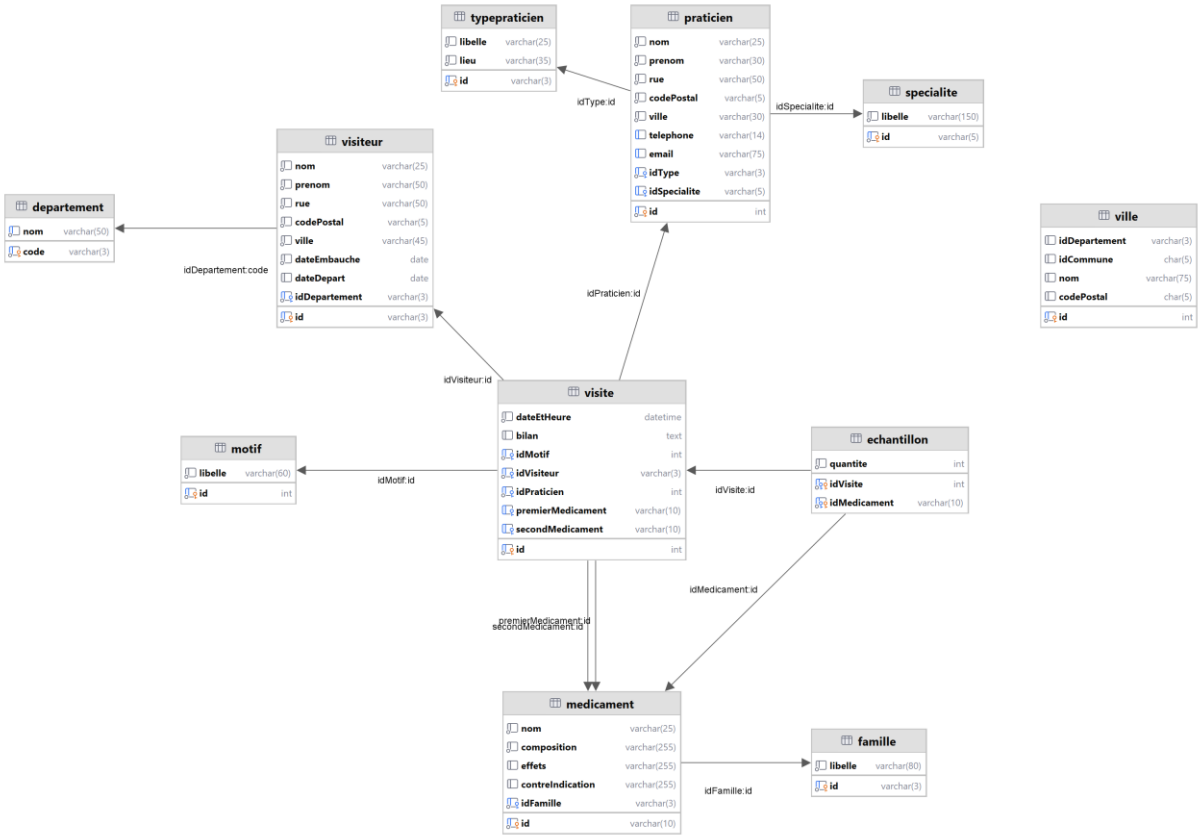
Table	Action	Lignes	Type	Interclassement
departement	Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_0900_ai_ci
echantillon	Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_0900_ai_ci
famille	Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_0900_ai_ci
medicament	Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_0900_ai_ci
motif	Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_0900_ai_ci
praticien	Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_0900_ai_ci
specialite	Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_0900_ai_ci
typepraticien	Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_0900_ai_ci
ville	Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_0900_ai_ci
visite	Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_0900_ai_ci
visiteur	Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_0900_ai_ci

Rappel sur la collation : Voir slam.lst > Base de données > Jeu de caractères et collation > Compatibilité

2. Réaliser la vue relationnelle sous PhpMyAdmin



3. Réaliser la vue relationnelle sous PhpStorm



3. Alimentation de la table département à partir de l'API https://geo.api.gouv.fr

4. Exécuter la requête https://geo.api.gouv.fr/departements depuis le navigateur

geo.api.gouv.fr/departements

Impression élégante


```
[
  {
    "nom": "Ain",
    "code": "01",
    "codeRegion": "84"
  },
  {
    "nom": "Aisne",
    "code": "02",
    "codeRegion": "32"
  },
  {
    "nom": "Allier",
    "code": "03",
    "codeRegion": "84"
  },
  {
    "nom": "Alpes-de-Haute-Provence",
    "code": "04",
    "codeRegion": "93"
  },
  {
    "nom": "Hautes-Alpes",
    "code": "05",
    "codeRegion": "93"
  },
  {
    "nom": "Alpes-Maritimes",
    "code": "06",
    "codeRegion": "93"
  },
  {
    "nom": "Ardèche",
    "code": "07",
    "codeRegion": "84"
  },
```


Étude du script **'Importation/departement.php'**

Ce script permet d'interroger l'API https://geo.api.gouv.fr/departements afin de récupérer la réponse dans le format json.

Il se connecte à la base de données GSB et parcourt la réponse renvoyée par l'API afin de générer les ordres SQL d'insertion dans la table departement.

Les départements d'outre-mer ne sont pas repris (longueur du code égal à trois)

Une explication s'impose sur la fonction
runInTransaction($db, function (PDO $db) use ($departements) {…})

function (PDO $db) : C'est une closure, c'est-à-dire une fonction anonyme.

Elle prend $db comme argument, ce qui correspond à $callback($db) dans runInTransaction.
use ($departements) : Permet à la fonction anonyme d'accéder à des variables qui sont en dehors de son scope (ici $departements).
Le corps { ... } Contient le code à exécuter dans la transaction.

5. Lancer le script **'Importation/departement.php'**
6. Vérifier le contenu de la table (Fenêtre Database>gsb>tables>departement – double clic (96 enregistrements)

	code	nom
1	01	Ain
2	02	Aisne
3	03	Allier
4	04	Alpes-de-Haute-Provence
5	06	Alpes-Maritimes

4. Alimentation de la table ville à partir de l'API https://geo.api.gouv.fr

7. Exécuter la requête https://geo.api.gouv.fr/communes?fields=nom,code,codesPostaux,codeDepartement&format=json&code=75056


```
[
  {
    "nom": "Paris",
    "code": "75056",
    "codesPostaux": [
      "75001",
      "75002",
      "75003",
      "75004",
      "75005",
      "75006",
      "75007",
      "75008",
      "75009",
      "75010",
      "75011",
      "75012",
      "75013",
      "75014",
      "75015",
      "75016",
      "75017",
      "75018",
      "75019",
      "75020",
      "75116"
    ],
    "codeDepartement": "75"
  }
]
```


Étude du script 'importer ville.php'

Le script interroge l'API pour récupérer la réponse au format JSON.

Il se connecte ensuite à la base de données GSB et parcourt les données afin de générer les instructions SQL d'insertion dans la table ville.

Le traitement est un peu plus complexe, car la propriété codePostal est un tableau : une ville peut donc posséder plusieurs codes postaux.

Il est donc nécessaire d'itérer sur cette propriété pour créer autant d'enregistrements qu'il y a de codes postaux.

8. Compléter la partie manquante du script et exécuter le script
9. Vérifier le contenu de la table ville (35 210 enregistrements)

WHERE idCommune = 75056 ORDER BY

	id	idDepartement	idCommune	nom	codePostal
1	29583	75	75056	Paris	75001
2	29584	75	75056	Paris	75002
3	29585	75	75056	Paris	75003
4	29586	75	75056	Paris	75004
5	29587	75	75056	Paris	75005
6	29588	75	75056	Paris	75006
7	29589	75	75056	Paris	75007
8	29590	75	75056	Paris	75008
9	29591	75	75056	Paris	75009
10	29592	75	75056	Paris	75010

5. Alimentation des autres tables (sauf visite)

Nous disposons dans le sous-répertoire data de l'ensemble des données récupérés de l'existant dans le format Json, Il s'agit donc de lire ces fichiers, de récupérer les données sous forme de tableau et de parcourir ces tableaux pour lancer les requêtes 'Insert'
Le script fonction.php comporte les fonctions permettant de réaliser ce travail
Le script 'importer famille.php' réaliser l'importation de familles.

10. En vous aidant du script 'importer famille.php', compléter les autres scripts : motif, medicament, praticien, specialite, typepraticien et visiteur
11. Vérifier le contenu des tables correspondantes (famille : 20, médicament : 28, motif : 5, praticien : 107, specialite : 44, typePraticien : 5)

En analysant ces scripts, on peut envisager de généraliser le traitement, il faut pour cela paramétrer :

- le nom de la table à mettre à jour : 'motif' par exemple,
- le nom et l'emplacement du fichier json : 'data/motif.json'
- la liste des colonnes de la table : ['id', 'libelle']

LA requête

- r à rendre la requêparamétrable au niveau de la liste des colonnes : $sql = "insert into `$table` ($listeColonne) values ($lesValeurs)";

Il faudrait fournir la liste des colonnes le nom de la table, le nom du fichier json et la liste des colonnes ou la récupérer à partir de la première ligne des fichiers json

12. Compléter et lancer le fichier importer_json.php

Remarque : il est possible de se passer de la liste des colonnes car on peut la récupérer à partir de la première ligne des fichiers json

6. Alimentation de la table visite

13. Lancer le script visite.sql afin de créer quelques visites en tenant compte de la date actuelle

Remarque : le script visite devra être relancé pour avoir un jeu d'essai utilisable à l'instant t.

Puisque nous n'avons pas encore mis en place les triggers, il faut vérifier que les données générées respectent les conditions demandées :

Cohérence entre praticien et visiteur : dans le même département
Prévue dans plus d'une heure mais dans les 2 moins à venir
Pas le dimanche
Entre 8 et 19 heures
Une seule visite programmée par praticien
Au moins deux heures entre chaque visite d'un même visiteur (le plus dur)

14. Compléter le script 'Requête de contrôle sur les visites.sql'

7. Création des vues

Toutes les visites sont centralisées dans la table visite, or un visiteur ne doit avoir accès qu'à ses propres visites. De même il ne doit pouvoir travailler qu'avec ses praticiens, les villes de son département et les échantillons distribués
On ne peut donc pas donner des droits directement sur ces tables car cela permettrait à un visiteur de voir les visites des autres visiteurs.
Afin de limiter l'accès aux seules données du visiteur connecté à l'application, il faut mettre en place des vues sur lesquelles le visiteur aura des droits.
Chaque visiteur aura son propre compte sur le serveur MySQL qui correspond à son id, il faut créer une vue principale qui récupère l'id, le nom et le département du visiteur du visiteur connecté.
Pour connaitre l'utilisateur connecté, MySQL dispose de la fonction user().
Pour la comparaison avec l'id, il faut ajouter @localhost : concat(visiteur.id, '@localhost') = user() ;
Les autres vues utiliseront cette vue pour filtrer les données correspondantes au visiteur

15. Compléter le script vue.sql

Rappel : toutes les vues sont par défaut créées avec l'option SQL SECURITY DEFINER et s'exécute donc avec les droits du créateur (ici root) donc le visiteur n'a pas besoin de disposer des droits sur les tables mises en œuvre dans la vue. C'est ici une sécurité importante

8. Création des utilisateurs

Comme indiqué dans l'introduction, chaque visiteur doit correspondre à un utilisateur dans la base de données. Cela va permettre d'assurer la sécurité des accès, chaque visiteur ne pouvant voir que les données qui le concerne (ses visites, ses praticiens etc.)

Nous n'allons pas créer les utilisateurs un par un mais trouver une solution programmable.
Le langage de programmation MySQL offre la possibilité de mettre en place des curseurs
Pour cela une procédure createuser doit être mise en place, elle utilise un curseur qui va parcourir les visiteurs afin de générer l'instruction create user et les instructions Grant correspondantes

Chaque visiteur dispose des mêmes droits de consultation sur les tables 'stables' et les vues
Pour éviter de devoir attribuer individuellement ses droits à chaque utilisateur, nous allons créer un rôle à qui nous allons donner les droits de consultation et d'exécution des procédures stockées

La procédure createuser n'aura plus qu'à donner ce rôle à chaque utilisateur

La procédure pouvant être relancée, l'instruction drop user est préalablement exécuter.
Il n'est pas possible de lancer directement l'instruction create user lorsque cette dernière comporte des paramètres, il faut dans ce cas générer la commande à l'aide de la fonction concat qui va réunir les parties fixes et les parties variables puis d'utiliser les commandes prepare et execute

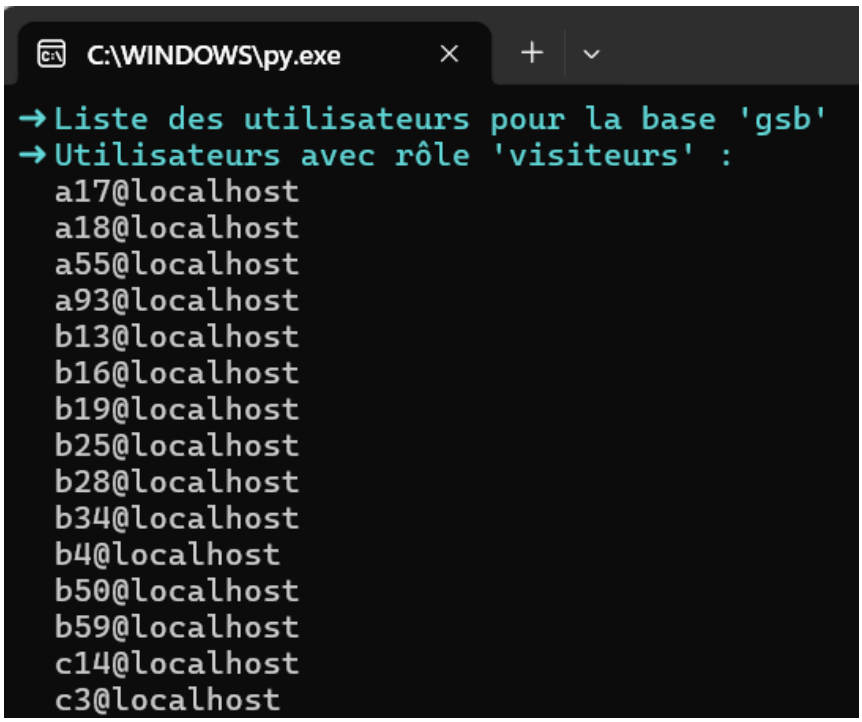
```
set @sql = concat('create user ', idVisiteur, '@localhost identified by '', mdp, '');
        prepare requete from @sql;
        execute requete;
```

Dans les dernières versions de MySQL, Lorsque vous configurez un rôle dans MySQL, il n'est actif que si :
L'utilisateur exécute explicitement la commande SET ROLE nonRole pour l'activer.
Un rôle par défaut a été défini pour cet utilisateur lors de la création ou après (alter user).

Le dossier Utilisateur contient un script create.php réalisant le même travail. L'avantage étant de pourvoir utiliser des structures itératives pour réaliser les ordres SQL.

Le dossier Script Python/Utilisateur comporte trois scripts permettant la création des utilisateurs, la consultation des utilisateurs et la suppression des utilisateurs.

16. Tester chaque version permettant la création des utilisateurs en exécuter les scripts dans l'ordre suivant : user.sql – get_user.py – dropt_user.py - get_suer.py |createUser.php – get_user.py – dropt_user.py – get_user.py | create_user.py – get_user.py



Avantage de la création d'un compte MySQL pour chaque visiteur

La protection est assurée au plus près de la donnée, donc impossible d'accéder à des données d'une autre visiteur

Critique sur la création d'un compte MySQL pour chaque visiteur

En production réelle : On évite de créer un compte MySQL par utilisateur métier.

L'authentification est gérée par l'application.

La base de données n'a qu'un seul compte applicatif.

Pourquoi :

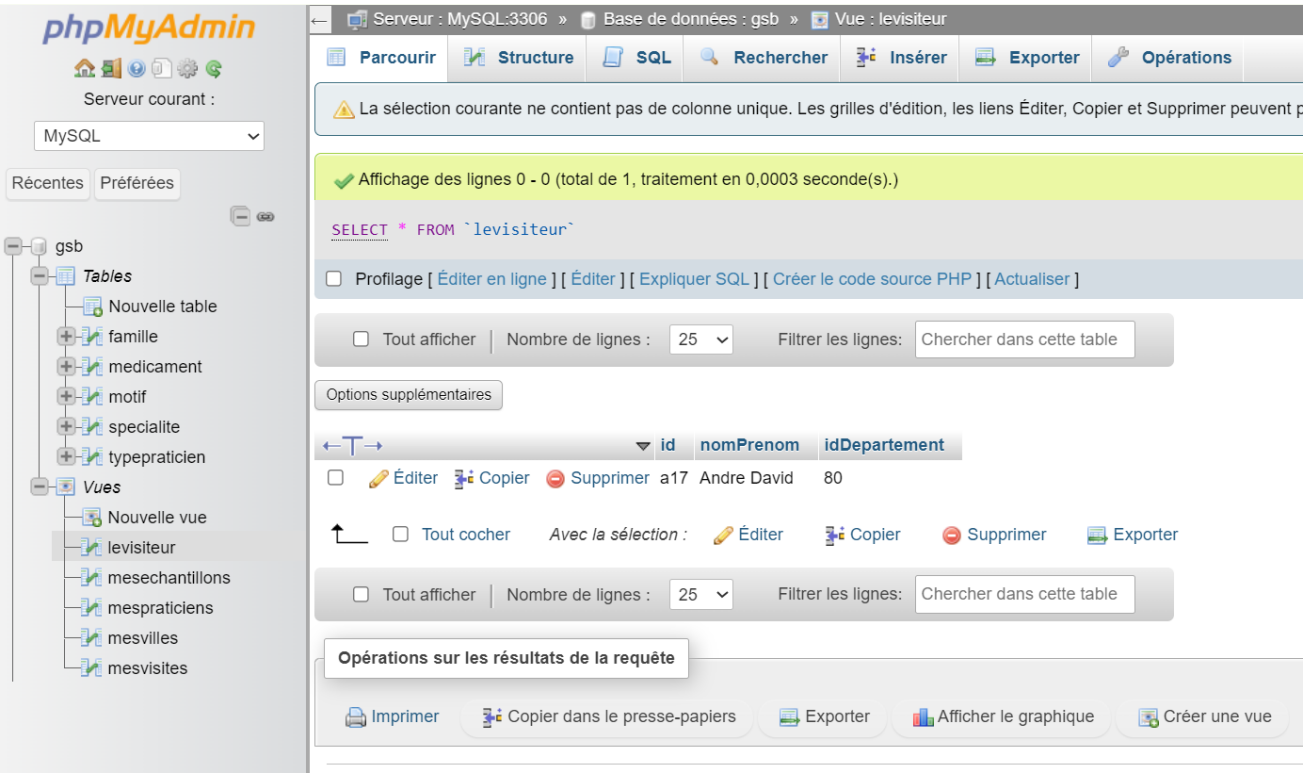
Scalabilité : avec des milliers d'utilisateurs, créer autant de comptes MySQL est inefficace et lourd à gérer.

Migration / hébergement : sur un serveur mutualisé, on ne peut pas créer dynamiquement des comptes MySQL, donc le modèle échoue.

Maintenance : changer un mot de passe utilisateur demanderait de modifier le compte MySQL, ce qui n'est pas pratique.

9. Vérification des accès utilisateur

17. Sous PhpMyAdmin, se connecter avec le compte a17 et le mot de passe 19910826



10. Sauvegarde de la base de données

Une fois la base mise en place, une sauvegarde doit être conservé afin d'éviter de devoir relancer l'ensemble des scripts.

La réalisation d'une exportation depuis phpMyAdmin est possible mais peut dans certains cas poser problème. Nous avons déjà rencontré le problème d'absence d'échappement des apostrophes dans les messages des triggers. Ce problème semble avoir disparu sur la version actuellement utilisée.

De plus il faudra préalablement supprimer la base de données avant de lancer la sauvegarde (ou ajouter drop database, create database à la main)

Nous disposons d'un script PHP sauvegarder.php dans le répertoire sauvegarde qui réalise ce travail en exécutant l'utilitaire mysqldump.exe.

18. Exécuter une première sauvegarde de la base de données

Remarque : La clause DEFINER est ajouté par défaut si la procédure ou fonction a été créée avec SQL SECURITY DEFINER, par contre la clause SQL SECURITY DEFINER n'est pas reprise car c'est la valeur par défaut.

Il n'existe pas d'option dans mysqldump.exe pour retirer la clause DEFINER, cela peut poser un problème si on utilise le script SQL sur un autre serveur qui n'a pas forcément l'utilisateur root.

Pour éviter ce problème le script relie le script SQL et retire la clause en utilisant la fonction :
preg_replace('/DEFINER=`[^`]+`@`[^`]+` /', '', $sqlContent);

```
// suppression de la clause DEFINER dans le fichier SQL
   $sqlContent = file_get_contents($outputFile);
   $sqlContent = preg_replace('/DEFINER=`[^`]+`@`[^`]+` /', '', $sqlContent);
   file_put_contents($outputFile, $sqlContent);
```

Autre solution : Le script python **save_one_database.py** permet de sauvegarder une base de données sélectionné parmi toutes les bases de données en réalisant le même travail (drop, create et definer)

Image de la base finale

Table	Action						Lignes	Type	Interclassement
departement	Parcourir	Structure	Rechercher	Insérer	Vider	Supprimer	96	InnoDB	utf8mb4_0900_ai_ci
echantillon	Parcourir	Structure	Rechercher	Insérer	Vider	Supprimer	5	InnoDB	utf8mb4_0900_ai_ci
famille	Parcourir	Structure	Rechercher	Insérer	Vider	Supprimer	20	InnoDB	utf8mb4_0900_ai_ci
levisiteur	Parcourir	Structure	Rechercher	Insérer	Éditer	Supprimer	~0	Vue	---
medicament	Parcourir	Structure	Rechercher	Insérer	Vider	Supprimer	28	InnoDB	utf8mb4_0900_ai_ci
mesechantillons	Parcourir	Structure	Rechercher	Insérer	Éditer	Supprimer	~0	Vue	---
mespraticiens	Parcourir	Structure	Rechercher	Insérer	Éditer	Supprimer	~0	Vue	---
mesvilles	Parcourir	Structure	Rechercher	Insérer	Éditer	Supprimer	~0	Vue	---
mesvisites	Parcourir	Structure	Rechercher	Insérer	Éditer	Supprimer	~0	Vue	---
motif	Parcourir	Structure	Rechercher	Insérer	Vider	Supprimer	5	InnoDB	utf8mb4_0900_ai_ci
praticien	Parcourir	Structure	Rechercher	Insérer	Vider	Supprimer	107	InnoDB	utf8mb4_0900_ai_ci
specialite	Parcourir	Structure	Rechercher	Insérer	Vider	Supprimer	44	InnoDB	utf8mb4_0900_ai_ci
typepraticien	Parcourir	Structure	Rechercher	Insérer	Vider	Supprimer	5	InnoDB	utf8mb4_0900_ai_ci
ville	Parcourir	Structure	Rechercher	Insérer	Vider	Supprimer	35 210	InnoDB	utf8mb4_0900_ai_ci
visite	Parcourir	Structure	Rechercher	Insérer	Vider	Supprimer	21	InnoDB	utf8mb4_0900_ai_ci
visiteur	Parcourir	Structure	Rechercher	Insérer	Vider	Supprimer	65	InnoDB	utf8mb4_0900_ai_ci